Design and Optimization of a Quadruped Jumping Controller

Contents

1 Introduction

In earlier practicals, we controlled a single leg in 2D. This project extends those concepts to a quadruped in 3D, where you will design and optimize a jumping controller inspired by the "Quadruped-Frog" paper [1]. You will learn how single-leg control principles generalize to multiple legs, and you will implement both a jumping controller and a continuous forward hopping controller, which serves as a first step toward quadruped locomotion in the next project.

1.1 Code Structure

The code is hosted at `https://gitlab.epfl.ch/lgevers/lr-miniproject-1`. The `README.md` gives details on installation and code structure.

1.2 Report Structure

Include a short report (no longer than 5 pages, excluding the generative AI declaration) including plots and results addressing the questions stated at the end of this file. You do not need to repeat the methodology that already exists in this file, rather your own methods and results that you generated. This project is 15% of the total grade and the report needs to be submitted by 21st October 23:59. Note that this is during the fall break and there will be no practical session that day.

2 Background

2.1 Quadruped Modeling

In this section we discuss the quadruped modeling used in pybullet. The quadruped has 4 legs, each with 3 joints (hip, thigh, calf), for a total of 12 motors. When querying joint states or sending torques to the motors, we thus have an array of length 12. The legs are numbered 0-3 in order Front Right (FR, 0), Front Left (FL, 1), Rear Right (RR, 2), Rear Left (RL, 3). The motor order is always hip, thigh, calf.

In many cases, we will be interested in the location of the foot in the leg frame. As an example, let's consider the position of the front right (FR) foot at a "neutral" position (i.e. the foot is directly under the hip) when standing at 0.25 m. The foot position in the leg frame is then $(x, y, z) = (0, \text{-hip\_length}, -0.25)$ m. Please think about whether the other legs will have the same leg frame coordinates (why is y for FR negative? What are the coordinates for our simulation?). The function `get_jacobian_and_position` will return this format.

On this note, recall from lecture the relationship between the joint angles $\boldsymbol{q}$ and foot position $\boldsymbol{p}$ for leg i:

$$\boldsymbol{p} = f(\boldsymbol{q}) \tag{1}$$

where the function $f(\cdot)$ denotes the forward kinematics of a single leg with respect to the leg frame. Differentiating this relationship gives the foot velocity $\boldsymbol{v}$:

$$\boldsymbol{v} = \boldsymbol{J}(\boldsymbol{q}) \cdot \dot{\boldsymbol{q}} \tag{2}$$

where $\boldsymbol{J}(\boldsymbol{q}) \in \mathbb{R}^{3\times3}$ is the single-leg Jacobian of the foot position w.r.t. the leg frame. The Jacobian can also be used to map a desired force $\boldsymbol{F}$ at the end effector (foot) to joint torques:

$$\boldsymbol{\tau} = \boldsymbol{J}^T(\boldsymbol{q})\boldsymbol{F} \tag{3}$$

These three relationships will be used repeatedly throughout this project.

2.2 Jumping Controller

Here, we will summarize the main architecture and components you will need to implement the jumping controller for this project. The high-level architecture is given in Figure 1, for more details we encourage you to refer to the original paper.

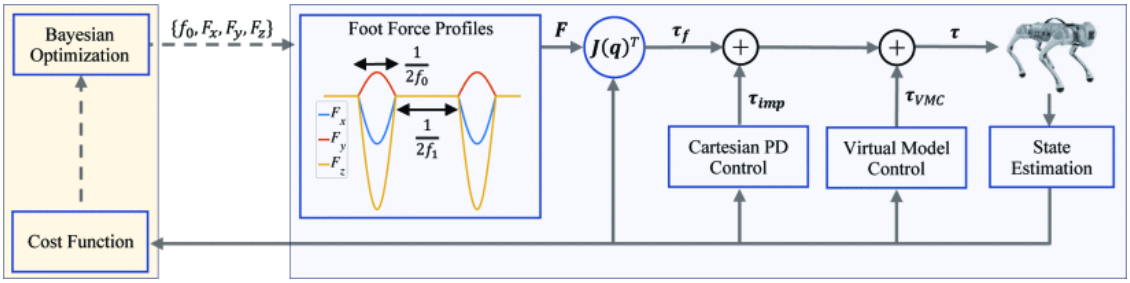

Figure 1: Jumping control architecture from [1].

The jumping controller works by designing force profiles to be applied at the feet, which is tracked at the joint level using the foot Jacobian. To generate a force profile where impulse and idle durations are different, we use an oscillator with two different frequencies:

$$\dot{\theta} = 2\pi f_i, \quad \text{where} \quad f_i = \begin{cases} f_0 & \text{if } \pi \le \theta < 2\pi, \\ f_1 & \text{if } 0 \le \theta < \pi \end{cases} \tag{4}$$

where θ is the phase of the oscillator, f_i the frequency of the oscillator, f_0 is the force impulse frequency and f_1 is the frequency between impulses. (Note that this is a differential equation for $\dot{\theta}$ which you will have to integrate in the code to obtain the phase θ.)

The phase θ determines the amplitude of the force profile at a given timestep as follows:

$$\boldsymbol{F}_i = \begin{cases} \begin{bmatrix} F_x & F_y & F_z \end{bmatrix}^\intercal \sin(\theta) & \text{if } \sin(\theta) < 0 \\ \boldsymbol{0}_{3\times 1} & \text{otherwise} \end{cases} \tag{5}$$

where $\boldsymbol{F}_i$ is the force to be applied at leg i, and $F_{\{x,y,z\}}$ are the peak force amplitudes in the x, y, z directions. To obtain omnidirectional jumping, it's useful to sometimes invert the sign of the force for certain legs before applying tracking it at the joint level (e.g., what direction should each leg push in for twisting?).

Note that on top of the force generated by the force profile, you should also have a gravity compensation force to account for the weight of the robot. The approach is similar to the second practical, although here the weight of the robot is distributed over 4 legs.

To keep the feet at a nominal position under the hip while on the ground and in the air, a Cartesian impedance controller is used. Remember from the earlier practicals that a typical Cartesian PD is implemented as follows for a single leg:

$$\boldsymbol{\tau}_{\text{Cartesian}} = \boldsymbol{J}^T(\boldsymbol{q}) \left[\boldsymbol{K}_{p,Cartesian} (\boldsymbol{p}_d - \boldsymbol{p}) + \boldsymbol{K}_{d,Cartesian} (\boldsymbol{v}_d - \boldsymbol{v}) \right] \tag{6}$$

where $\boldsymbol{p}_d$ is the desired foot position, $\boldsymbol{v}_d$ the desired foot velocity, $\boldsymbol{p}$ and $\boldsymbol{v}$ the current foot position and velocity, respectively. $\boldsymbol{J}(\boldsymbol{q})$ is the foot Jacobian at joint configuration $\boldsymbol{q}$, and $\boldsymbol{K}_{p,Cartesian}$ and $\boldsymbol{K}_{d,Cartesian}$ are diagonal matrices of proportional and derivative gains. This time you will apply it to 4 different legs, and the sign of the foot position in the y-direction might change depending on the leg. For a less springy behavior when landing after a jump you might want to add some damping to the legs on top of this Cartesian PD (which control will you use for this?).

To increase the stability of the base and avoid high Cartesian gains the controller adds Virtual Model Control (VMC) to regulate the base's roll and pitch angles, similar to [2,3]. As introduced in lecture 3 of the course, VMC uses virtual elements to keep the robot upright, and computes the necessary torques to replicate the forces generated by these virtual elements. Specifically, we attach virtual springs between the hypothetical XY plane aligned with the robot's base with a horizontal plane in world frame. These virtual springs generate forces to align the base parallel with the ground and reduce the pitch and roll angles.

The XY plane $\boldsymbol{P}$ through the robot base is obtained by rotating the coordinates of the corners of the plane through the body with the robot's base rotation matrix with respect to the world frame $\boldsymbol{R}$:

$$\boldsymbol{P} = \boldsymbol{R} \begin{bmatrix} 1 & 1 & -1 & -1 \\ -1 & 1 & -1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \tag{7}$$

$$\tag{8}$$

We then attach virtual springs between this XY plane and the horizontal plane in world frame:

$$\boldsymbol{F}_{\text{VMC}} = \begin{bmatrix} \boldsymbol{0}_{2\times 4} \\ k_{\text{VMC}} \left(\begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \boldsymbol{P} \right) \end{bmatrix} \tag{9}$$

where k_{VMC} is the gain. Each column of $\boldsymbol{F}_{\text{VMC},i}$ corresponds to the force to be applied at the foot of leg i:

$$\boldsymbol{\tau}_{\text{VMC},i} = \boldsymbol{J}_i(\boldsymbol{q}_i)^\intercal \boldsymbol{F}_{\text{VMC},i} \tag{10}$$

All the resulting desired torques coming from the force profiles, the gravity compensation, the impedance control, and the virtual model, are then added together for each leg and sent to the motors. Note that the resulting jumping behavior is determined by the force profile, parameterized by impulse and idle frequencies, and peak forces. You will first explore these parameters systematically (i.e., with parameter sweeps). Subsequently, these parameters will be optimized by an optimization framework, where you will have to define appropriate variables and cost functions to optimize different jumping behaviors.

3 Assignment

3.1 Controller

Implement the jumping controller by filling in the files `quadruped_jump.py` and `profiles.py` carefully. The lines you have to complete are tagged with `#TODO`. For more information on these files, refer to the code comments and the instructions in the `README.md`. Once you have a working jumping controller, address the questions below in your report.

3.1.1 Questions

1. Start with forward jumping. What nominal foot position seems to give the better performance? How does the jumping and stability compare between a low and high center of mass?
2. Describe how your omnidirectional jumping controller implements (a) forward jumping, (b) lateral jumping (left and right), and (c) twist jump (clockwise and counterclockwise). Additionally, explore the influence of relevant force profile parameters on each type of jumping performance using parameter sweeps.
3. How helpful is the VMC? What gains do you use? Illustrate the difference with a comparison between jumps with and without the VMC component (tip: the effect might be stronger after multiple jumps).

3.1.2 Tips

- Focus on one component at a time. Use the `on_rack=True` argument to keep the robot in the air to help verify your implementations before moving on to the next one. Use the `tracking_camera=False` argument to have a free camera you can move with the mouse to inspect the robot at different angles.
- Plotting individual components (e.g., the force profiles) is very helpful with debugging unintended behaviors.

3.2 Optimization

Once your controller is ready and stable (you might go back and forth to further tune your controller), you will optimize the variables controlling the jumping behavior using Bayesian optimization. Specifically, you will optimize the foot force profiles parameters. You are free to explore the optimization of other variables, but we encourage you to start with just the force profiles first. We provide a skeleton in `quadruped_jump_opt.py` using the Optuna library [4] which reuses the functions you have implemented in the previous section. Complete the `#TODO`s in the file in order to run the different optimizations and address the questions below in your report.

3.2.1 Questions

1. Optimize the furthest (a) forward jump, (b) lateral jump (either left or right), and (c) twist jump. Describe your optimization variables and objective functions to achieve these behaviors. Report the performance and variable values across runs for each behavior, using **no more than 50 trials**. Finally, record a video (either using `record_video=True` or screen recordings) of each best jump.
2. Describes the measures you took for the optimal solutions to have stable jumping and avoid falling.

3. Optimize the fastest forward continuous hopping controller. Describe your approach and report your results. Limit yourself to 50 trials. If you do intend on using multiple optimization runs (e.g., different seeds, different subset of variables), limit each of them to 50 trials. Record a video of your final controller.
4. Discuss: What considerations do you need to take into account before porting the controller from simulation to hardware? How would you run the optimization process on the hardware?
5. Discuss: What extra challenges do you think you will face when designing a walking controller (vs a hopping controller here)?

3.2.2 Tips

- If the optimization process struggles to find suitable solutions in less than 50 trials, your controller might be too unstable. Tune your controller further (e.g., gains) before resuming the optimizations.
- When reporting results, make sure to use multiple runs with different seeds and report the average and standard deviations of the runs.
- You won't be graded on the absolute performance of your hopping controller. We rather want to see a systematic approach to the design and optimization of the controller. Tuning at the finest level to squeeze out the absolute best performance is therefore not necessary, 50 trials with a well-tuned controller should be more than enough (and in fact can be done in 20). You are also free (and encouraged) to try and explore extensions not described here, even if it fails (but with a good motivation behind your approach).

3.3 Submission

When submitting zip your report, your code, and report, together named `lr_mp_group_{group_number}.zip` with your group number. Your code should include **code files only** (e.g., no `venv` directory for the virtual environment). You should include the `env` folder and the `requirements.txt` in case you are adding new dependencies. Videos should be named with descriptive names (e.g., `forward_jump.MP4`) that you refer to in your report. The structure of your submission folder should look like this:

```
lr_mp1_group_{group number}.zip
|- report.pdf               # Your report in PDF
|- env/                     # Unmodified
|  |- __init__.py
|  |- utils.py
|  \- simulation.py
|- videos/                  # Only include videos that are relevant to your report!
|  |- forward_jump.MP4      # Descriptive name example
|  \- ...
|- quadruped_jump.py        # Script with your changes
|- quadruped_jump_opt.py    # Script with your changes
|- profiles.py              # Script with your changes
\- requirements.txt         # Dependencies
```

Note that the max submission size is 100 MB, which means you may have to compress your videos.

3.3.1 Tips

- For writing a good report, it is helpful to assume that the reader has never seen the report before. Ideally, they should be able to understand the flow and logic of the report just by reading it.
- Organize the content clearly with a logical structure, consized explanations and clear visuals to guide the reader.
- Remember to proof-read the report before submission.

A Generative AI Declaration

The use of generative AI tools for this project is allowed (although optional), given an explicit declaration on its use. Please include a *Generative AI declaration* section in your report if you have used any form of generative AI throughout this project, and answer the following questions. If the use of AI differs heavily between team members, add one declaration per team member.

1. Which aspects of the project did you (attempt to) use generative AI for? This can be anything ranging from code completion, report writing, or even clarifying concepts of the course. Describe your overall strategy of when to use or not to use AI.
2. In which aspects was generative AI the **most** useful? Illustrate with a prompt you used and the resulting answer.
3. In which aspects was generative AI the **least** useful? Illustrate with a prompt you used and the resulting answer.
4. Discuss: the use of generative AI has/has not made this project smoother to complete.
5. Discuss: the use of generative AI has/has not had an impact on my ability to learn and understand the concepts behind this project (can be positive or negative).

References

[1] G. Bellegarda, M. Shafiee, M. E. Özberk, and A. Ijspeert, "Quadruped-Frog: Rapid Online Optimization of Continuous Quadruped Jumping," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 1443–1450.

[2] J. Pratt, C.-M. Chew, A. Torres, P. Dilworth, and G. Pratt, "Virtual Model Control: An Intuitive Approach for Bipedal Locomotion," *The International Journal of Robotics Research*, vol. 20, no. 2, p. 129–143, 2001.

[3] M. Ajallooeian, S. Pouya, A. Sproewitz, and A. J. Ijspeert, "Central Pattern Generators augmented with virtual model control for quadruped rough terrain locomotion," in *2013 IEEE International Conference on Robotics and Automation*, 2013, pp. 3321–3328.

[4] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, p. 2623–2631.